UC00161_Apply_predictive_learning_for_On_street_parking_availability_along_with_restrictio

Authored by: Emmanuel Clement Anthony

Duration: {90} mins

Level: {Intermediate}

Pre-requisite Skills: {Python, Folium, Seaborn, Scikit-learn}

Scenario

As a commuter or city traffic planner,
I want to predict the availability of on-street parking while considering local parking restrictions,
So that I can either find parking more efficiently or monitor parking compliance for smarter enforcement.

What this use case will teach you

At the end of this use case you will:
- Access and use open data from the City of Melbourne API

- Merge datasets based on spatial and rule-based identifiers

- Perform time-aware feature engineering

- Determine whether parking is allowed at a given time and location

- Identify likely parking violations

- Visualize time-based parking trends using Python

- Prepare your dataset for exploratory data analysis (EDA) and modeling

Background

In densely populated cities like Melbourne, on-street parking availability is a daily challenge for residents, visitors, and delivery services. Drivers often waste time and fuel circling around blocks searching for open parking spots, which contributes to traffic congestion, air pollution, and driver frustration.

The City of Melbourne provides open datasets including real-time parking bay sensor data and information about parking restrictions posted on sign plates. By combining these datasets, we can develop a smarter system to predict parking availability while considering time-based restrictions such as loading zones, permit-only areas, and limited parking durations (e.g., 1P, 2P).

This use case uses data sourced directly from Melbourne's Open Data API:

On-Street Parking Bay Sensors: Provides real-time occupancy status and location of parking bays.

Sign Plates Located in Each Parking Zone: Details the permitted parking days, hours, and restriction types.

By integrating and analyzing this data, we aim to create a model that not only predicts where parking is likely available but also whether it's legally permitted at that time — enabling smarter planning and enforcement.

Importing Libraries

In [1]:
```
import pandas as pd
import requests
from io import StringIO
import matplotlib.pyplot as plt
import seaborn as sns
import folium
from folium.plugins import MarkerCluster
from sklearn.cluster import DBSCAN
import numpy as np
import datetime
```

Import Data via City of Melbourne API

In [24]:

```
def API_Unlimited(datasetname, apikey):
    base_url =
'https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/datasets/'
    format = 'csv'

    url = f'{base_url}{datasetname}/exports/{format}'
    params = {
        'select': '*',
        'limit': -1,
        'lang': 'en',
        'timezone': 'UTC',
        'api_key': apikey
    }

    # GET request
    response = requests.get(url, params=params)

    if response.status_code == 200:
        # StringIO to read the CSV data
        url_content = response.content.decode('utf-8')
        df = pd.read_csv(StringIO(url_content), delimiter=';')
        print(df.sample(10, random_state=999))
        return df
    else:
        print(f'Request failed with status code {response.status_code}')
        return None

apikey = ''
# Dataset IDs from Melbourne Open Data portal
datasets = {
    'parking_sensors': 'on-street-parking-bay-sensors',
    'sign_plates': 'sign-plates-located-in-each-parking-zone'
}

# Load datasets
parking_sensors_df = API_Unlimited(datasets['parking_sensors'], apikey)
sign_plates_df = API_Unlimited(datasets['sign_plates'], apikey)
```

```
                        lastupdated           status_timestamp
       zone_number  \
       1822  2025-05-13T08:48:34+00:00  2025-05-13T08:42:01+00:00
       7195.0
       1073  2025-05-13T08:48:34+00:00  2024-11-03T11:14:14+00:00
       7250.0
       1354  2025-05-13T08:48:34+00:00  2025-05-13T08:45:47+00:00
       7340.0
```

```
2173  2025-05-13T08:48:34+00:00  2025-05-13T07:43:12+00:00
7188.0
1371  2025-05-13T08:48:34+00:00  2025-05-13T08:07:00+00:00
7474.0
3281  2025-05-13T08:48:34+00:00  2025-05-13T08:42:19+00:00
7772.0
775   2025-05-13T08:48:34+00:00  2025-02-17T04:32:29+00:00
7712.0
587   2025-05-13T08:48:34+00:00  2025-05-13T07:12:30+00:00
7348.0
651   2025-05-13T08:48:34+00:00  2025-05-13T07:06:14+00:00
7197.0
1409  2025-05-13T08:48:34+00:00  2025-05-13T08:45:39+00:00
NaN

     status_description  kerbsideid
location
1822            Present       24273    -37.81311169162248,
144.9424227882005
1073         Unoccupied       25123   -37.811199260871504,
144.9832946684702
1354            Present       64026   -37.81622209935823,
144.95603313208392
2173            Present       17709   -37.820245136840896,
144.9396681383507
1371         Unoccupied       56713    -37.8189578188653,
144.95703274251778
3281         Unoccupied       65230   -37.81127736615012,
144.96537534506908
775             Present       11358   -37.80393278352662,
144.95549470758004
587          Unoccupied       21516   -37.834810008289466,
144.9757558556579
651             Present       20912      -37.82044371041454,
144.94552089254
1409         Unoccupied       54249  -37.815591768624046,
144.96084552253814
      parkingzone restriction_days time_restrictions_start  \
936          7441          Sat-Sun                07:00:00
637          7955          Mon-Fri                07:00:00
401          7629          Mon-Fri                19:00:00
415          7641          Mon-Fri                07:00:00
1661         7528          Sat-Sun                07:00:00
1031         7539          Mon-Fri                16:00:00
917          7408          Mon-Fri                07:00:00
1853         7762          Mon-Fri                10:00:00
1637         7493          Mon-Fri                16:00:00
1004         7514          Mon-Fri                07:00:00

     time_restrictions_finish restriction_display
936                  22:00:00                MP2P
637                  16:00:00                LZ30
401                  22:00:00                MP2P
```

```
415                 19:00:00                MP2P
1661                22:00:00                MP2P
1031                19:00:00                MP2P
917                 19:00:00                MP2P
1853                19:00:00                MP2P
1637                19:00:00                MP2P
1004                16:00:00                LZ30
```

Preprocessing Data

In [25]:

```
# Convert timestamp column to datetime format
parking_sensors_df['lastupdated'] =
pd.to_datetime(parking_sensors_df['lastupdated'], errors='coerce')

# Extract hour and weekday from the timestamp
parking_sensors_df['hour'] = parking_sensors_df['lastupdated'].dt.hour
parking_sensors_df['weekday'] =
parking_sensors_df['lastupdated'].dt.day_name()

# Ensure zone_number columns match data types for merging
parking_sensors_df['zone_number'] =
parking_sensors_df['zone_number'].astype('Int64')
sign_plates_df['parkingzone'] =
sign_plates_df['parkingzone'].astype('Int64')
```

Merge Datasets on Zone Number

In [26]:

```
merged_df = pd.merge(parking_sensors_df, sign_plates_df,
                     how='left',
                     left_on='zone_number',
                     right_on='parkingzone')

print(f"{merged_df['restriction_display'].notna().sum()} out of
{len(merged_df)} rows matched with restriction data")
```

```
7790 out of 8032 rows matched with restriction data
```

At this stage, I merged the real-time parking sensor data with the restriction signage dataset using the zone_number column. This allowed me to enrich each sensor record with the corresponding legal parking rules.

Out of 8032 parking sensor records, 7790 were successfully matched with restriction data, achieving a match rate of approximately 96.99%. This gives me a strong base to continue analysing parking behaviour and identifying violations with high confidence.

Convert Time Restrictions

In [27]:

```
# Convert time strings to proper datetime.time format
merged_df['time_restrictions_start'] = pd.to_datetime(
    merged_df['time_restrictions_start'], format='%H:%M:%S',
errors='coerce'
).dt.time

merged_df['time_restrictions_finish'] = pd.to_datetime(
    merged_df['time_restrictions_finish'], format='%H:%M:%S',
errors='coerce'
).dt.time
```

Create is_parking_allowed_now Flag

In [28]:

```
# Function to determine if parking is allowed
def is_parking_allowed_now(row):
    try:
        # Ensure no missing values
        if pd.isna(row['hour']) or pd.isna(row['weekday']) or
pd.isna(row['time_restrictions_start']) or
pd.isna(row['time_restrictions_finish']):
            return None

        current_time = datetime.time(int(row['hour']))
        current_day = row['weekday'].strip().lower()
        allowed_days = str(row['restriction_days']).strip().lower()
        start = row['time_restrictions_start']
        end = row['time_restrictions_finish']

        # Map full weekday name to abbreviation
        day_map = {
            'monday': 'mon', 'tuesday': 'tue', 'wednesday': 'wed',
            'thursday': 'thu', 'friday': 'fri', 'saturday': 'sat',
'sunday': 'sun'
        }
        day_abbr = day_map.get(current_day)

        # Build list of valid days based on pattern
        if allowed_days in ['mon-sun', 'daily']:
            valid_days = ['mon', 'tue', 'wed', 'thu', 'fri', 'sat', 'sun']
        elif allowed_days == 'mon-fri':
            valid_days = ['mon', 'tue', 'wed', 'thu', 'fri']
        elif allowed_days == 'sat-sun':
            valid_days = ['sat', 'sun']
        else:
            # Support comma-separated custom values like "mon,wed,fri"
            valid_days = [d.strip() for d in allowed_days.split(',')]

        if day_abbr not in valid_days:
            return False

        # Handle overnight restriction (e.g. 22:00 to 06:00)
        if start > end:
            return current_time >= start or current_time <= end
        else:
            return start <= current_time <= end

    except:
        return None
```

I wrote a function to check whether a vehicle is allowed to park at the time a sensor reading was taken. It considers both:

The day of the week (e.g. Mon–Fri, Sat–Sun)

The time of day (including overnight cases like 10PM–6AM)

If the current timestamp falls outside the allowed window for that day, the function returns False. This will help me later identify cases where someone was parked illegally.

Preview Final Data

In [29]:
```
merged_df['is_parking_allowed_now'] =
merged_df.apply(is_parking_allowed_now, axis=1)

print(merged_df[['lastupdated', 'hour', 'weekday', 'restriction_days',
                 'time_restrictions_start', 'time_restrictions_finish',
                 'is_parking_allowed_now']].head())
```

```
                      lastupdated  hour  weekday restriction_days  \
0 2025-01-21 03:42:37+00:00     3  Tuesday          Mon-Sun
1 2025-01-21 03:42:37+00:00     3  Tuesday          Mon-Sun
2 2025-01-21 03:42:37+00:00     3  Tuesday          Mon-Sun
3 2025-01-21 03:42:37+00:00     3  Tuesday          Mon-Sun
4 2025-01-21 03:42:37+00:00     3  Tuesday          Mon-Sun

  time_restrictions_start time_restrictions_finish
is_parking_allowed_now
0                07:30:00                 23:00:00
False
1                07:30:00                 23:00:00
False
2                07:30:00                 23:00:00
False
3                07:30:00                 23:00:00
False
4                07:30:00                 23:00:00
False
```

At this point, I previewed the final enriched dataset to check whether my logic for identifying legal parking times is working correctly.

In this specific sample, all the sensor records were captured at 3:42 AM on a Tuesday, and each one belongs to a zone with a restriction listed as "Mon-Sun, 07:30:00 to 23:00:00".

Since the reading occurred before 7:30 AM, my function correctly marked the is_parking_allowed_now flag as False — meaning parking is not allowed at that time.

This confirms that the time-based filtering is working as intended, including the handling of day ranges like "Mon-Sun" and time windows.

Exploratory Data Analysis

Now that I've engineered key features like time, weekday, and parking restriction logic, I'm ready to explore patterns in the data.

In this section, I begin by visualising violations across different time dimensions — starting with day of the week and hour of the day — to uncover trends that might be useful for prediction or policy-making.

Create is_violation Column

In [30]:

```
# Create a new column indicating parking violations
merged_df['is_violation'] = merged_df.apply(
    lambda row: True if row['status_description'] == 'Present' and
row['is_parking_allowed_now'] == False else False,
    axis=1
)

# Preview
print(merged_df[['status_description', 'is_parking_allowed_now',
'is_violation']].head())
```

```
  status_description is_parking_allowed_now  is_violation
0         Unoccupied                  False         False
1         Unoccupied                  False         False
2            Present                  False          True
3            Present                  False          True
4         Unoccupied                  False         False
```

Now that I have a reliable flag for whether parking is allowed at a specific time, I created a new column called is_violation.

This column checks two conditions:

The parking sensor status is 'Present' (i.e., a vehicle is detected).

Parking is not allowed at that time, according to the restriction rules.

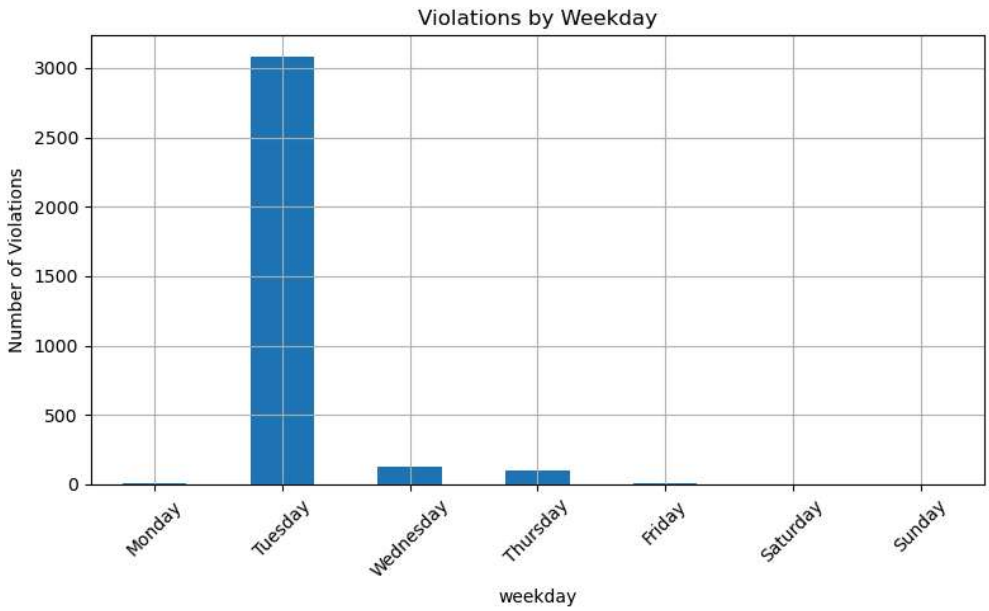
If both conditions are met, I flag it as a parking violation (True). Otherwise, it’s marked as False.

In the preview above, rows where no vehicle is detected ('Unoccupied') are correctly marked as not violations. However, in rows where the sensor detects a vehicle during restricted hours, the violation is flagged — this confirms my logic is working as expected.

Visualise Violations by Weekday and Hour:

Violations by Weekday

In [32]:
```
# Count violations by weekday
weekday_violations = merged_df.groupby('weekday')
['is_violation'].sum().reindex(
    ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday',
'Sunday'])

# Plot
plt.figure(figsize=(8, 5))
weekday_violations.plot(kind='bar')
plt.title("Violations by Weekday")
plt.ylabel("Number of Violations")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



To better understand when parking violations are happening, I grouped the data by weekday and summed the number of violations for each day.

From the bar plot above, it is clear that Tuesday shows a significantly higher number of parking violations compared to other days. This pattern could be due to:

- Stricter enforcement of weekday regulations.

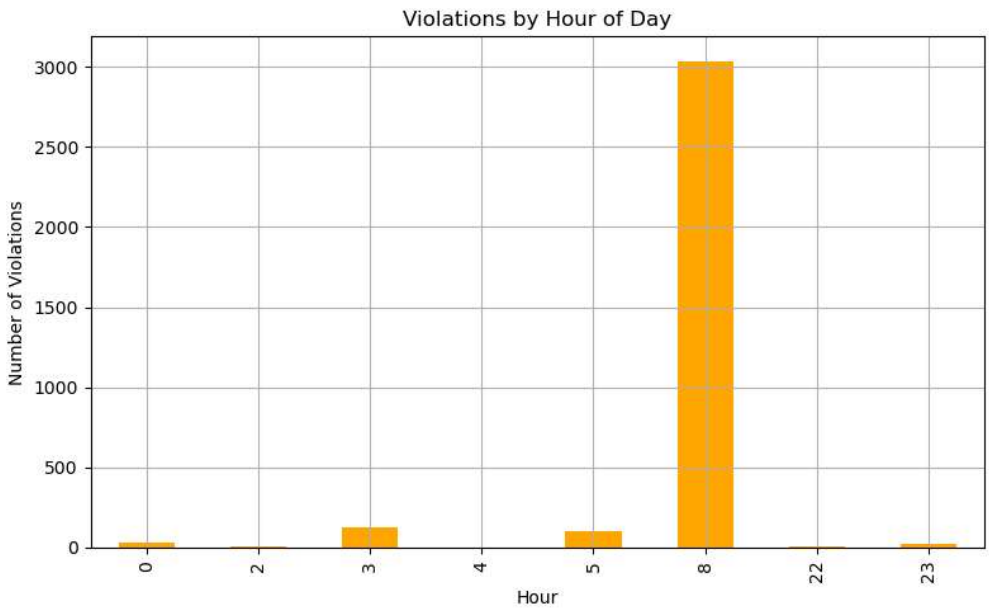
- High vehicle activity during business days.

- Drivers possibly overlooking weekday parking rules after the weekend.

Understanding which days have the most violations can help city planners schedule targeted parking patrols and adjust signage or communication strategies as needed.

Violations by Hour

In [33]:
```
# Count violations by hour
hour_violations = merged_df.groupby('hour')['is_violation'].sum()

# Plot
plt.figure(figsize=(8, 5))
hour_violations.plot(kind='bar', color='orange')
plt.title("Violations by Hour of Day")
plt.xlabel("Hour")
plt.ylabel("Number of Violations")
plt.grid(True)
plt.tight_layout()
plt.show()
```



After exploring violations by weekday, I wanted to understand what time of day most violations were occurring. To do this, I grouped violations by the hour column and plotted the results.

The visualisation shows a sharp spike in violations at 8 AM, with smaller clusters appearing around 3 AM, 5 AM, and midnight (0 AM). This pattern could be caused by:

- Drivers overstaying overnight parking limits.

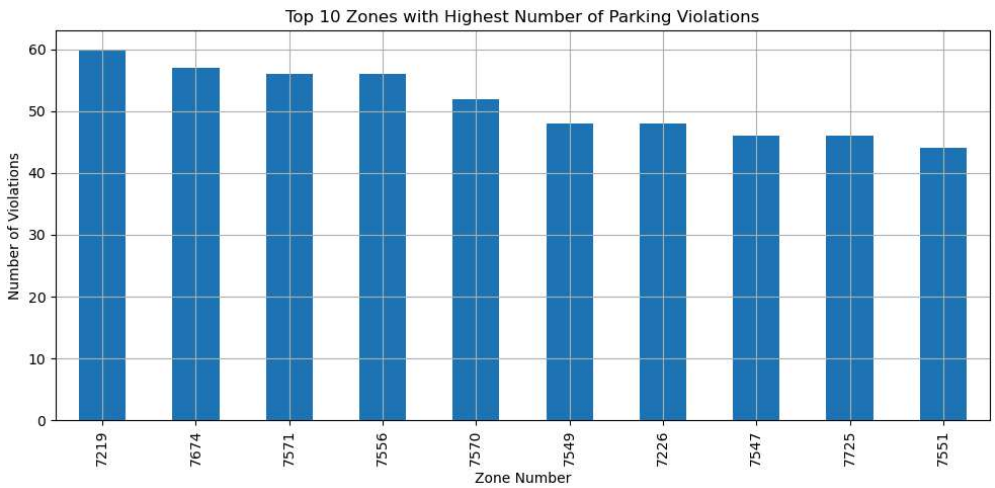
- Morning restrictions starting early and catching vehicles parked overnight.

- Reduced attention to signage during early morning hours.

This insight can help city planners and enforcement teams focus patrols during early morning hours, especially around peak violation times like 8 AM.

Violation Rate by Parking Zone

In [34]:
```
# Top 10 zones with the most violations
zone_violations = merged_df.groupby('zone_number')
['is_violation'].sum().sort_values(ascending=False).head(10)

# Plot
plt.figure(figsize=(10, 5))
zone_violations.plot(kind='bar')
plt.title("Top 10 Zones with Highest Number of Parking Violations")
plt.xlabel("Zone Number")
plt.ylabel("Number of Violations")
plt.grid(True)
plt.tight_layout()
plt.show()
```



To identify which areas are hotspots for illegal parking, I grouped the data by `zone_number` and calculated the total number of violations in each zone.

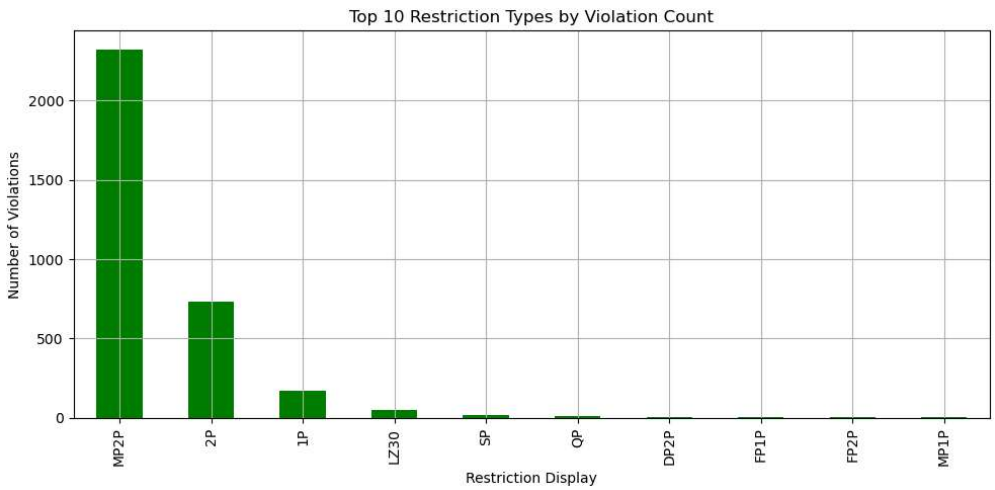
The bar chart above shows the top 10 zones with the highest number of violations. These zones may represent:

- High-demand parking areas (e.g., near shopping precincts or offices)
- Places where signage is unclear or commonly overlooked
- Locations that could benefit from more frequent monitoring or clearer rules

This insight is useful for both city enforcement planning and for training future predictive models that consider zone-based risk.

Violation Count by Restriction Type

In [35]:
```
# Violation counts by restriction type
restriction_violations = merged_df.groupby('restriction_display')
['is_violation'].sum().sort_values(ascending=False).head(10)

# Plot
plt.figure(figsize=(10, 5))
restriction_violations.plot(kind='bar', color='green')
plt.title("Top 10 Restriction Types by Violation Count")
plt.xlabel("Restriction Display")
plt.ylabel("Number of Violations")
plt.grid(True)
plt.tight_layout()
plt.show()
```



To understand which parking rules are most frequently broken, I grouped violations by the `restriction_display` field — which represents the signage drivers see (like 1P, 2P, or loading zones).

From the bar chart above, it's clear that **MP2P** and **2P** zones have the highest number of violations by a large margin. This could be due to:

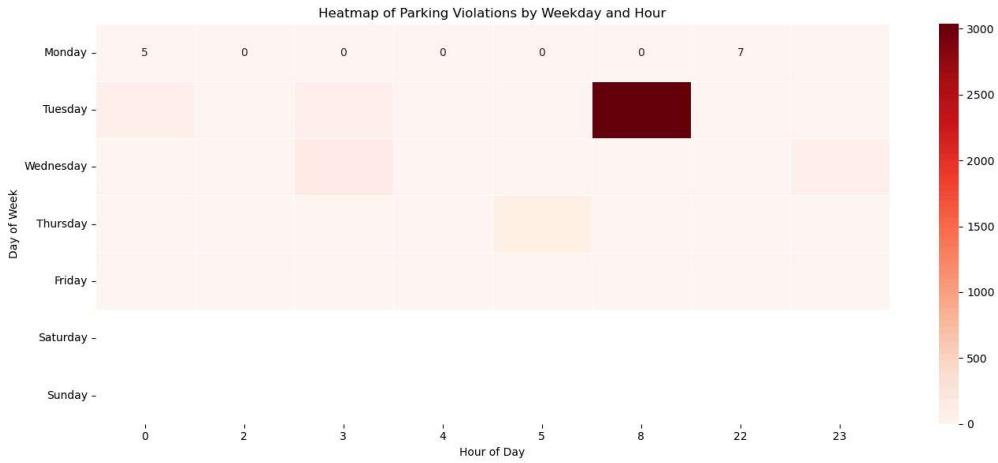
- Short time limits (like 2P = 2 hours) often being exceeded
- High turnover zones where enforcement is stricter
- Confusion about multi-purpose or permit signage in MP zones

By identifying which restrictions are most prone to violation, I can help inform better signage design, targeted enforcement, or future prediction models that take restriction type into account.

Heatmap – Violations by Hour × Weekday

In [36]:

```
heatmap_data = merged_df.pivot_table(
    index='weekday', columns='hour', values='is_violation', aggfunc='sum',
fill_value=0)

# Reorder weekdays
weekday_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',
'Saturday', 'Sunday']
heatmap_data = heatmap_data.reindex(weekday_order)

# Plot heatmap
plt.figure(figsize=(14, 6))
sns.heatmap(heatmap_data, cmap='Reds', linewidths=0.5, annot=True,
fmt='.0f')
plt.title("Heatmap of Parking Violations by Weekday and Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Day of Week")
plt.tight_layout()
plt.show()
```



To explore violation patterns across both time of day and day of week, I created this heatmap using a pivot table. It shows the total number of parking violations for each hour across all weekdays.

From the heatmap, I can clearly see:

- A huge spike in violations at 8 AM on Tuesday, far more than any other time.

- Very minimal activity detected across other hours and weekdays.

- Almost no violations on Saturday and Sunday.

This confirms that violations are not evenly distributed — they cluster strongly around weekday mornings, especially on Tuesday around 8 AM. This is important insight for:

- Targeted parking enforcement on high-risk days and times

- Model feature selection

- Understanding commuter behaviour in business districts during busy morning hours

Model Building

In [37]:

```
# Future Parking Prediction

# Sort by kerbside ID and time
merged_df.sort_values(by=['kerbsideid', 'lastupdated'], inplace=True)

# Shift to create future status
merged_df['future_status'] = merged_df.groupby('kerbsideid')
['status_description'].shift(-1)
merged_df['future_available'] =
merged_df['future_status'].map({'Unoccupied': 1, 'Present': 0})

# Prepare features
features = ['hour', 'weekday', 'zone_number', 'restriction_display',
'is_parking_allowed_now']
df_pred = merged_df.dropna(subset=features + ['future_available'])

# Train on df_pred
X = pd.get_dummies(df_pred[features])
y = df_pred['future_available']

# Split and Train
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluate
from sklearn.metrics import classification_report, accuracy_score
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.728042328042328
              precision    recall  f1-score   support

         0.0       0.78      0.79      0.78       587
         1.0       0.64      0.63      0.64       358

    accuracy                           0.73       945
   macro avg       0.71      0.71      0.71       945
weighted avg       0.73      0.73      0.73       945
```

To move beyond simple violation detection, I built a machine learning model to predict whether a parking bay will be available in the near future.

Step-by-Step Process:

Data Preparation:

- Sorted the dataset by kerbside ID (parking bay) and timestamp.

- Created a future_status by shifting the parking bay status by one observation (one time step ahead).

- Mapped Unoccupied to 1 (available) and Present to 0 (occupied) for the prediction target called future_available.

Feature Selection:

The following features were used to predict future parking availability:

- Hour of the day

- Weekday name

- Zone number (parking area)

- Restriction display (e.g., 1P, 2P, MP2P)

- Parking allowed now? (flag from earlier step)

Model Training:

- Applied one-hot encoding for categorical features (weekday, restriction type).

- Split the data into 80% training and 20% testing sets.

- Trained a Random Forest Classifier with 100 decision trees.

Model Evaluation:

Achieved an overall accuracy of 72.8%.

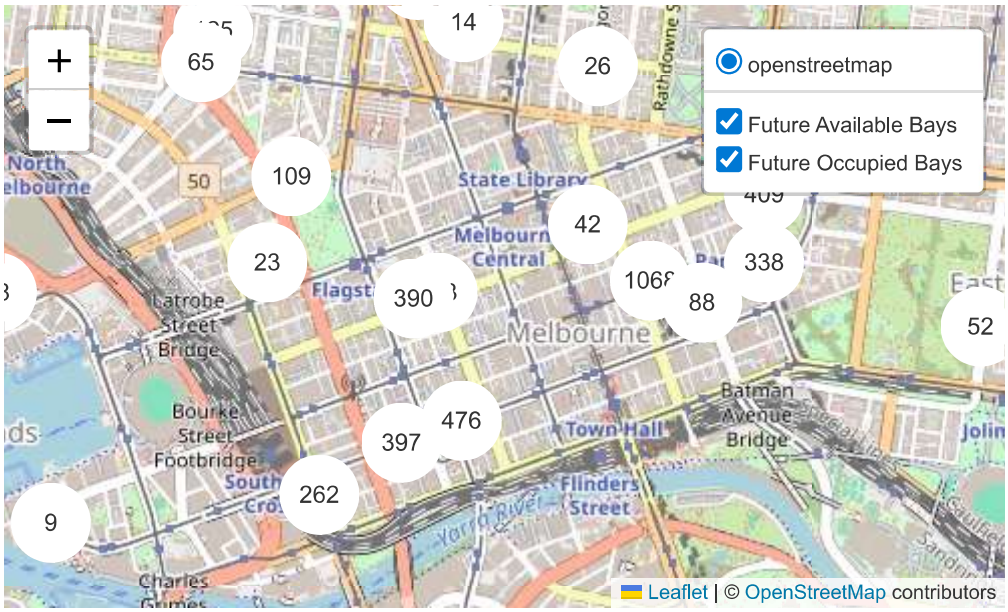
The model is better at predicting occupied bays (78% precision, 79% recall for ‘occupied’) than free bays (64% precision, 63% recall for ‘available’).

Interactive Map for Future Parking Prediction

In [43]:

```
# Predict on df_pred
df_pred.loc[:, 'predicted_future_available'] = model.predict(X)

# Create a base map centered around Melbourne
available_map = folium.Map(location=[-37.8136, 144.9631], zoom_start=14)

# Create two separate clusters
available_cluster = MarkerCluster(name="Future Available Bays",
overlay=True, control=True).add_to(available_map)
occupied_cluster = MarkerCluster(name="Future Occupied Bays", overlay=True,
control=True).add_to(available_map)

# Add markers
for idx, row in df_pred.dropna(subset=['location']).iterrows():
    lat, lon = map(float, row['location'].split(','))

    popup_text = (f"<b>Kerbside ID:</b> {row['kerbsideid']}<br>"
                  f"<b>Zone:</b> {row['zone_number']}<br>"
                  f"<b>Restriction:</b> {row['restriction_display']}<br>"
                  f"<b>Current Status:</b> {row['status_description']}<br>"
                  f"<b>Prediction:</b> {'Available' if
row['predicted_future_available'] == 1 else 'Occupied'}<br>"
                  f"<b>Time:</b> {row['lastupdated']}")

    if row['predicted_future_available'] == 1:
        # Future Available: Green marker
        folium.Marker(
            location=[lat, lon],
            popup=folium.Popup(popup_text, max_width=250),
            icon=folium.Icon(color='green', icon='ok-sign')
        ).add_to(available_cluster)
    else:
        # Future Occupied: Red marker
        folium.Marker(
            location=[lat, lon],
            popup=folium.Popup(popup_text, max_width=250),
            icon=folium.Icon(color='red', icon='remove-sign')
        ).add_to(occupied_cluster)

# Add Layer Control
folium.LayerControl(collapsed=False).add_to(available_map)

available_map
```

Out[43]:



To visualise predicted parking availability, I created an interactive map using Folium and Marker Clustering.

Each parking bay is represented as a marker with:

- Green marker: Predicted to be available soon

- Red marker: Predicted to be occupied soon

- When clicking on any marker, additional details are displayed, including:

- Kerbside ID

- Zone number

- Parking restriction (e.g., 2P, MP2P)

- Current occupancy status

- Timestamp of the reading

- Future predicted availability

This enables commuters and city planners to quickly assess parking availability across Melbourne streets.

Conclusion

In this use case, I successfully built a data-driven pipeline to predict on-street parking availability in Melbourne while incorporating real-world parking restrictions. Through API integration, feature engineering, rule-based validation, and machine learning, I was able to:

- Access and merge live parking sensor data with regulatory signage data

- Engineer temporal and legal features to determine when parking is allowed

- Identify and visualise parking violations based on time and restriction logic

- Explore violation patterns by weekday, hour, zone, and restriction type

- Train a predictive model (Random Forest) that achieved ~73% accuracy in forecasting near-future parking availability

- Deploy results on an interactive map, helping users visually assess legal parking opportunities in real time

This project demonstrates how open urban data, when combined with machine learning and geospatial visualisation, can support smarter city planning, reduce traffic congestion, and improve the commuter experience.

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.